Color codes used in this documentation: -> unavailable; -> Please note; ->WIP(Work in Progress)

This is a work in progress documentation, containing all input file's structure, values for each parameter used in json based input files, it will be updated along with new developments.

Best practices of coding and naming conventions requested to be followed (update in current code) :

- Wherever possible please try to refrain from using snake_case. Software, Multiphysics team agreed upon using camelCase for variable names, thus must be strictly followed to avoid future confusions.
- Read and strictly follow the C++ best practices guidebook: https://lefticus.gitbooks.io/cpp-best-practices/content/03-Style.html
- Use "UpperCamelCase" for class names and "lowerCamelCase" for objects.

Solver Input File (solver.json)

Solver input file has 4 parameters, each solver Input file must contain following parameters:

- *solver* = Information regarding type of solver, each problem will contain only one solver parameter.
- *equation* = Information regarding the type of equation we intend to solve, it can contain multiple equations. (i.e. in case of coupled problems simulation)
- *materialProperty* = Material property variables and constants of the domain. This can also contain multiple material parameters depending on how many types of material domain contain. (domain with multiple materials to be tested).
- *optimizationConfig* = Topology optimization related input variables, it can contain multiple optimization parameters. This should be defined only when solving topology optimization problems

```
{
    "solver":
    [
        {
                "coordinateSystem" : <string>,
                "type" : <string>,
                "startTime" : <double>,
                "endTime" : <double>,
                "timeStep" : <double>,
                "stoppingCriteria" : <double>,
                "massMatrixType" : <boolean>,
                "theta" : <double> ,
                "algorithm" : <string>
        }
    ],
```

coodinateSystem : Global coordinate system can be only one of the following options : "AXIS" (Axisymmetric 2D) or "1D" or "2D" or "3D"

type : "transient" or "steady",

// All variables for "solver" parameter, below to be defined only when –> type = transient

startTime : Starting time for time step of a time dependent problems

endTime : End time step for a transient problems

timeStep : Each time step size

stoppingCriteria : Convergence criteria for the problem defined by user {usually very small value << 1}

massMatrixType : a Boolean variable ; 0 = Lumped Mass matrix; 1 = Consistent Mass Matrix

theta : Generalized alpha time discretization algorithm; can take on of the 4 plausible values defined below
0 ->Forward Euler; 1 -> Backward Euler; 2.0/3.0 -> Galerkin; 1.0/2.0 -> Crank-Nickolson

algorithm : Type of algorithm to solve the linear system of equations, only one of the three options
Classical ; Quantum; Hybrid (classical+Quantum)

```
"equation":
[
    {
        "name" : <string>,
        "type" : <string>,
        "meshFile" : <string>,
        "meshField" : <string>,
        "materialPropertyName" : <string>, <array for multi-material domain>
        "feShape" : <string>,
        "gaussPoints" : <int>
    }
],
```

name : A user defined name as identifier for equation

type : Type of equation “heatTransfer”, “planeStress”, “planeStrain”, “linearElastic3D”, “optimization”

// if simulationType (project) → Solid & solver → “coordinateSystem” = “2D”
type : “planeStress” or “planeStrain”

// if simulationType (project) → Solid & solver → “coordinateSystem” = “3D”
type : “linearElastic3D”

meshFile : mesh file name

meshField : physical field name within mesh where the equation must be applied.

materialPropertyName : Name of material property which correspond to this equation, to be used for linking the equations with material properties

In case of multiple materials “**materialPropertyName**” would be an **array** containing all the names of material properties name defined in the materialProperty parameter.

feShape : Type of finite element selected by user for descretization of the domain; to be as per problem
1D elements = “2NodeRod”, “3NodeRod”, “2NodeBeam”, “3NodeBeam”
2D elements = “3NodeTri”, “6NodeTri”, “4NodeQuad”, “9NodeQuad”
3D elements = “8NodeHexa”, “4NodeTetra”

gaussPoints : Number of Gauss points for each element

```
"materialProperty" :
[
    {
        "name" : <string>,
        "type" : <string>,
                                    // Conditional parameters dependent on “type”
        "youngsMod" : <double>,
        "nu" : <double>,
        “thermalConductivity” : <array> or <double>,
        "thickness": <double>,
        "rho" : <double>,
        “specificHeat” : <double>,
                                    // Defined only in case of multi-material domain
        “meshField” : <sting>
    },
]
```

name : Text name given by user for identification of different types of material, also links with *materialPropertyName* in equations.

type : Material type currently = “LinearElastic” or “HeatTransfer-iso” or “HeatTransfer-aniso”

// Thickness to be defined only for 2D cases

thickness : Domain thickness to be defined for only when: solver→ coordinateSystem = 2D.

// If material type = LinearElastic

YoungsMod : Youngs modulus of stiffness based on material
NU : Poinson’s Ratio

// if material Type = HeatTransfer-iso or HeatTransfer-aniso

rho : Mass Density
specificHeat *: Specific heat of material*
thermalConductivity : The thermal conductivity of the material

// If type = HeatTransfer-iso

“**thermalConductivity**” = <double> i.e single value for isotropic material

//if type = *HeatTransfer-aniso*

“**thermalConductivity**” = <array of double> for isotropic material and array for anisotropic material

############# NOTE #############
only isotropic material available for alpha version

// if domain contains multiple materials

meshField: physical name of the material region to which this material property must be applied

*# **NOTE (Software Team)**: The thermal conductivity array size would change as per the problem (domain) material type and coordinateSystem (defined within solver parameters) i.e.::*

If materialProperty → “type” = “HeatTransfer-aniso” then :
In case of 2D problems “thermalConductivity” = [Kxx, Kxy, Kyx, Kyy]. (size 4)
In case of 3D problems “thermalConductivity” = [Kxx, Kxy, Kxz, Kyx, Kyy, Kyz, Kzx, Kzy, Kzz]. (size 9)

// If topology optimization enabled / selected, only then optimizationConfig will have sub parameters

```
"optimizationConfig":
[
    {
        "minimize": <stiring>,
        "filterRadius": <double>,
        "maxIteration": <int>,
                        // Conditional parameters dependent on what must be minimized
        "volumeFraction": <double>,
        "maxMeanCompliance": <double>,
        ~~"fB": <double>~~
    }
]
```

minimize : Quantity that must be minimized, it should be one of following options: “compliance”, “volume” or “displacement”

filterRadius : adjacent element Filter radius to be used for density-based filtering

maxIterations : Maximum number of QGA iterations (total number of generation)

// If minimize = “volume”

volumeFraction : Percentage of material and void in domain.

// if minimize = “compliance”

maxMeanCompliance : Maximum mean compliance of the whole structure

~~**fB**~~ : ~~Float to binary conversion ratio for the QGA~~ (*Depricated*)

Parts input file contains info regarding the (domain) mesh file and accepted mesh file formats. The “parts” parameter can contain multiple mesh files in case of MBD or if domain is an assembly constituting of multiple parts.

Parts Input file (parts.json)

```
"parts":
 [
        {
                "name" : <string>,              // text box in GUI
                "format" : <string>,            // Drop down menu in GUI
                "meshFileName": <string>        // Upload button in GUI
        }
 ]
```

name : Text name to identify the part {in case of multibody and big assemblies}

format : The mesh file format currently only gmsh (.msh), in future hypermesh would be added.

meshFileName : is the mesh file name with extension that contains the subpart of assemble or Multi body

The boundary.json input file must contain 4 parameters, each parameter can have multiple quantities, pertaining to types of boundary conditions of the problem:

- *neumannBC* : This parameter contains the variables and constants as per different neumann boundaries defined in mesh, these are quantities that must be weakly enforced.
- *dirichletBC* : These are the boundary conditions that must be strongly enforced.
- *nonDesignDomain* : If there is any non-design domain defined in geometry for topology optimization, if there is no non-Design domain then keep this parameter empty.
- *InitalBC* : Initial state conditions to be defined for only transient problems i.e. if solver → type → "transient"

Boundary Input File (boundary.json)

```
{
    "neumannBC":
        [
            {
                "name": <string>,
                "boundaryType": <string>,
                "type": <string>,
                "variable": <string>,
                "meshFile": <string>,
                "meshField": <string>,
                "value": <scalar> ,
                "values" : <vector> or <array of scalar or array of vectors>,
                "filename": <filePath>
                                // Conditional parameters dependent on variable
                "convectionCoeff" : <double>,
                "ambientTemp" : <double>,
                "Q" : <double>,
                "domainSource" : <boolean>
            },
```

name: The parameter can take any string. This name will help the user to distinguish between multiple boundary conditions.

boundaryType: This parameter can take "point", "components", "normalToBoundary", "gradX", "gradY", "gradZ"

type: The type of the variable i.e. **t**his parameter can take "constant", "unifomlyDistributed", "temporal", "constantNodalFileInput", "temporalNodalFileInput"

variable: The variable identifies the quantity that must be weakly imposed on the neumann boundary. This parameter can take "force", "pressure/traction", "convectiveHeatTransfer", "heatFlux", "heatSourceSink"

meshFile: meshFile name

meshField: physical name of the boundary to which the boundary condition is being applied

// If variable = "temperature"

value: It is a scalar value, hence must be defined in "value" Scalar variable (i.e. single value)

// If variable = “traction” || “pressure” || “force” && boundaryType = “components” || “point”
values: Values will contain vector components of tractions/pressure/force i.e. [value1, value2, value3]

// If variable = “heatFlux”
values : Heat flux (vector) going through the boundary i.e. [value1, value2, value3]
sign convention for heatFlux → -ve means heat flowing into surface
+ve means heat flowing out of surface

// If variable = “convectiveHeatTransfer”
convectionCoeff : Constant coefficient of convection

ambientTemp : Free flow convection Temperature

// If variable = “heatSourceSink”
Q : Constant internal Heat generation (+value=source and -value=sink) per unit volume

domainSource : A boolean flag that identifies if source/sink is present at whole domain or if there is line/point source/sink present in domain

fileName: This parameter is used for the parameter type *FileInput. This contains the file path. The data inside the file can be scalar or vector and will in the order of the node in the physical field. (Skip for alpha version)

#***NOTE***: boundaryType = “point” signifies a point boundary, for example point loads.
Change “values” array input to be parsed from value1, value2, value3 as per backend discussion.

```
"dirichletBC":
[
    {
        "name": <string>,
        "type": <string>,
        "variable": <string>,
        “boundaryType”: <string>,
        "meshFile": <string>,
        "meshField": <string>,
        "value": <scalar>,
        “values”: <array> [value1, value2, value3]
    }
]
```

name: The parameter can take any string. This name will help the user to distinguish between multiple boundary conditions.

type : This parameter can take “constant”, “temporal”, “constantNodalFileInput”, “temporalNodalFileInput”

boundaryType : This parameter can be “point”, “line”, “surface”

variable: Identifies the variable that must be strongly imposed on the dirichlet boundary. This parameter can take “dispX”, “dispY”, “dispZ”, “velX”, “velY”, “velZ”, “temperature”, “bodyForce”

meshFile: meshFile name

meshField: physical name of the boundary to which the boundary condition is being applied

value: value will be scalar or array, it will be scalar if the type is constant. It will be array if the type is temporal. This parameter is not used for the parameter type *FileInput. Instead include the fileName parameter.

values: If variable == “bodyForce” then, it’s the **array** containing “bodyForce” component values of each nodal point in domain, i.e. in the form of [bodyForceX, bodyForceY, bodyForceZ]

*# **<u>NOTE (MPS)</u>**: Change values array input to be parsed from value1, value2, value3 as per backend discussion.*

```
"nonDesignDomain":
[
    {
        "name": <string>,
        "type": <string>,
        "meshFile": <string>,
        "meshField": <string>,
        "passiveValue": <int>
    }
]
```

name: This is a name assigned by the user in GUI

type: Type of boundary I.e. point, surface

meshFile : Mesh file path and name where non-design domain is defined

meshField : Mesh field name defined in mesh file to identify the non-design elements.

passiveValue : A passive value to be assigned to the defined non-design domain elements.

```
"initialBC":
[
    {
        "name": <string>,
        "type": <string>,
        "variable": <string>,
        "meshFile": <string>,
        "meshField": <string>,
        "value": <scalar/array>
        “filename”: <filePath>
    },
```

name: The parameter can take any string. This name will help the user to distinguish between multiple boundary conditions.

type: This parameter can take “constant {1}”, “nodalFileInput {2}”

variable: This parameter can take “dispX”, “dispY”, “dispZ”, “velX”, “velY”, “velZ”, “temperature”, “bodyForce”

meshFile: meshFile name

meshField: physical name of the boundary to which the boundary condition is being applied

value: value will be scalar it will be scalar for the type of constant. This parameter is not used for the parameter type nodalFileInput. Instead include the fileName parameter.

values: It is the array containing “bodyForce” values (if any) at each nodal point in domain, in the form of [bodyForceX, bodyForceY, bodyForceZ]

fileName: This parameter will be scalar used for the parameter type nodalFileInput. This contains the file path. The data inside the file will be scalar and will in the order of the node in the physical field. (Skip for alpha version)